BAB I
PENDAHULUAN

Bab ini menjelaskan dasar pengerjaan Tugas Akhir. Pembahasan mencakup latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan.

I.1 Latar Belakang

Large language model (LLM) dapat membantu pengembang membuat kode dari instruksi bahasa alami. Namun, kode yang menghasilkan keluaran sesuai permintaan belum tentu memenuhi kebutuhan keamanan. Sebagai contoh, fungsi untuk mengambil data dapat berjalan dengan benar, tetapi tetap membuka peluang injeksi SQL apabila masukan pengguna tidak ditangani dengan tepat. CyberSecEval melaporkan bahwa model yang diuji menghasilkan saran kode rentan pada rata-rata sekitar 30% kasus dalam benchmark tersebut (Bhatt et al., 2023). Hasil ini menunjukkan perlunya pemeriksaan keamanan sebelum kode hasil LLM digunakan. Persentase tersebut merupakan hasil pada benchmark dan model yang diuji, sehingga tidak berlaku untuk seluruh kode hasil LLM.

Salah satu cara memeriksa kode adalah *static application security testing* (SAST), yaitu pemeriksaan tanpa menjalankan program. SAST dapat menemukan pola kelemahan melalui aturan yang telah ditentukan. Akan tetapi, peringatannya masih dapat berupa *false positive*, yaitu temuan yang sebenarnya tidak menunjukkan kerentanan. LLM dapat membantu menilai peringatan dengan membaca konteks kode. QASecClaw menggunakan pendekatan ini dan melaporkan penurunan false positive yang disertai penurunan recall (Ameen et al., 2026). Hal tersebut menunjukkan bahwa penyaringan peringatan juga dapat menghilangkan sebagian temuan yang benar.

Pendekatan lain membagi pekerjaan kepada beberapa agen. AutoSafeCoder menghubungkan agen pembuat kode, reviewer, dan fuzzing dalam proses perbaikan berulang (Nunez et al., 2024). Multi-LLMSecCodeEval juga membandingkan kerja

sama beberapa model dan alat analisis statis pada tahap generasi, deteksi, serta perbaikan kode (Sabir et al., 2026). Kedua penelitian tersebut menjadi dasar pengembangan alur pemeriksaan kode hasil LLM. Penggunaan beberapa agen dan perbaikan berulang sudah diteliti, sehingga kontribusi penelitian perlu dijelaskan melalui cara komponen bekerja dan hasil evaluasinya.

Dalam alur pemeriksaan tersebut, hasil antaragen dan alat dapat berbeda. Reviewer dapat menilai suatu peringatan tidak relevan, sedangkan alat masih melaporkan kelemahan. Pengujian juga dapat gagal dijalankan karena lingkungan yang tidak sesuai. Keadaan tersebut memerlukan aturan tindak lanjut yang jelas. Refute-or-Promote menunjukkan pentingnya pemeriksaan empiris: salah satu kandidat kelemahan yang didukung beberapa reviewer dibantah melalui pengujian (Agarwal, 2026). Dengan demikian, alasan yang diberikan agen perlu diperiksa bersama bukti yang tersedia sebelum menjadi dasar keputusan.

Penelitian Beyond QASecClaw turut menguji hubungan penilaian model dengan bukti alat. Pada pengaturan Java yang diuji, penambahan konteks agen tidak memberikan manfaat yang konsisten dibandingkan pemeriksaan deterministik. Studi tersebut juga menyoroti kesesuaian bukti dengan program dan kondisi yang diperiksa (Al Mamun et al., 2026). Temuan ini memperkuat kebutuhan untuk mencatat asal bukti dan menguji manfaat tambahan agen. Hasilnya tetap dibatasi oleh konstruksi eksperimen dan model yang digunakan dalam penelitian tersebut.

Selain perbedaan hasil, reviewer dapat menerima teks yang sengaja diarahkan untuk memengaruhi penilaiannya. Komentar pada kode, misalnya, dapat berisi pernyataan bahwa kode telah diaudit dan boleh diterima. Sanitizer dapat menyiapkan masukan untuk mengurangi teks semacam ini. Namun, manfaatnya perlu diuji. Thornton menemukan bahwa pengaruh komentar adversarial tidak signifikan pada pengujian berpasangan, sedangkan penghapusan komentar dapat menurunkan kemampuan deteksi model yang lebih lemah (Thornton, 2026). Oleh karena itu, evaluasi pertahanan perlu tetap memperhatikan kemampuan menemukan kelemahan pada kode tanpa serangan.

Berdasarkan permasalahan tersebut, Tugas Akhir ini mengusulkan perancangan dan implementasi *multi-agent security gate* dengan *Evidence-Based Arbiter*. Gate ditempatkan setelah generator untuk memeriksa kode sebelum diteruskan. Agen bertugas menganalisis temuan, melakukan triase, menyusun pengujian, dan memeriksa kelengkapan bukti. Arbiter non-LLM menerapkan kebijakan untuk menentukan apakah kode diterima (*pass*), diperbaiki (*revise*), atau ditahan untuk ditinjau manusia (*block*). Kode yang diperbaiki oleh Fixer kembali menjalani pemeriksaan. Setiap keputusan dihubungkan dengan versi kode, hasil alat, hasil pengujian, dan aturan yang digunakan.

Fokus penelitian adalah ketepatan keputusan penerimaan kode ketika bukti tidak lengkap atau bertentangan, termasuk setelah perbaikan dan ketika reviewer menerima komentar manipulatif. Pengujian akan membandingkan gate dengan pemeriksaan yang lebih sederhana, serta mengukur dampak sanitizer dan aturan arbiter. Jumlah kode rentan yang lolos perlu dinilai bersama penerimaan kode tidak rentan, kasus yang ditahan, keberhasilan patch, dan biaya penggunaan model. Dalam Tugas Akhir ini, verifikasi keamanan berarti pemeriksaan terhadap persyaratan dan kategori kelemahan yang ditetapkan. Label lolos gate menunjukkan bahwa pemeriksaan tersebut terpenuhi, dengan cakupan sesuai pengujian yang dilakukan.

I.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah Tugas Akhir ini adalah sebagai berikut.

1. Bagaimana merancang dan mengimplementasikan multi-agent security gate dengan Evidence-Based Arbiter yang menggunakan bukti pemeriksaan untuk menentukan keputusan pass, revise, atau block, termasuk ketika bukti tidak lengkap atau bertentangan?
2. Bagaimana ketepatan keputusan gate dan hasil perbaikan kode dibandingkan dengan SAST saja serta SAST dengan satu reviewer LLM, ditinjau dari kode rentan yang lolos, penerimaan kode tidak rentan, fungsi kode, dan biaya pemeriksaan?

3. Bagaimana pengaruh sanitizer dan aturan arbiter terhadap ketahanan keputusan gate ketika reviewer menerima komentar manipulatif, serta terhadap kemampuan pemeriksaan pada kode tanpa serangan?

I.3 Tujuan Tugas Akhir

Tugas Akhir ini bertujuan mengembangkan dan mengevaluasi sistem pemeriksaan keamanan kode hasil LLM. Tujuan yang sesuai dengan rumusan masalah adalah sebagai berikut.

1. Menghasilkan implementasi gate yang menghubungkan agen pemeriksa, alat analisis statis, pengujian, dan perbaikan kode dengan arbiter non-LLM. Implementasi menyediakan aturan tindak lanjut untuk bukti yang belum lengkap atau bertentangan, serta mencatat dasar setiap keputusan.
2. Mengukur ketepatan keputusan dan hasil perbaikan gate dibandingkan dengan pembanding yang ditetapkan. Pengukuran mencakup tingkat kode rentan yang lolos, tingkat penerimaan kode tidak rentan, hasil deteksi, keberhasilan patch yang tetap memenuhi uji fungsi, jumlah kasus yang ditahan, penggunaan token, dan waktu pemrosesan.
3. Menilai perubahan keputusan pada pasangan kode tanpa serangan dan kode dengan komentar manipulatif. Evaluasi membandingkan penggunaan sanitizer dan aturan arbiter dengan kondisi ketika masing-masing komponen tersebut dihilangkan atau diganti oleh keputusan reviewer.

I.4 Batasan Masalah

Ruang lingkup Tugas Akhir ini dibatasi sebagai berikut.

1. Objek pengujian berupa fungsi atau potongan kode Python hasil LLM. Pengujian menggunakan tugas yang diambil dari SecurityEval dan subset Python LLMSecEval (Siddiq & Santos, 2022; Tony et al., 2023). Rancangan awal difokuskan pada CWE-22, CWE-78, CWE-89, CWE-502, dan CWE-798. Pemilihan sampel mempertimbangkan kelengkapan label acuan, aturan pemeriksaan, dan dukungan pengujian; sampel yang dikeluarkan dicatat beserta alasannya.

2. Sistem menggunakan model LLM yang telah tersedia, tanpa melatih atau melakukan fine-tuning model. Identitas model, versi prompt, parameter pemanggilan, serta versi dan aturan alat dicatat pada konfigurasi eksperimen.
3. Analisis statis menggunakan Semgrep dan Bandit. Pengujian fungsi serta keamanan dilakukan dalam lingkungan terisolasi untuk sampel yang memenuhi kebutuhan lingkungan. Pemeriksaan yang tidak berjalan dicatat sebagai tidak teruji dan menghalangi pass jika termasuk pemeriksaan wajib.
4. Kebijakan arbiter mencakup penerimaan, permintaan perbaikan, dan penahanan kode. Keputusan block merupakan permintaan tinjauan manusia; proses peninjauan oleh pengguna di lingkungan produksi berada di luar pengujian ini. Batas awal remediasi adalah dua ronde perbaikan setelah keluaran pertama.
5. Skenario manipulasi reviewer dibatasi pada perubahan atau penyisipan komentar yang mempertahankan perintah dan perilaku program. Penyerang tidak mengubah aturan arbiter, prompt sistem, hasil alat, atau knowledge store. Sanitizer menghasilkan tampilan khusus reviewer dan mempertahankan kode asli untuk pemindaian serta eksekusi.
6. Apabila contoh pola dan patch digunakan dalam knowledge store, isinya berasal dari data pengembangan dan dibekukan sebelum pengujian akhir. Patch serta label sampel uji tidak digunakan sebagai contoh retrieval untuk sampel yang sama.
7. Evaluasi dilakukan pada lingkungan eksperimen yang terkontrol. Hasilnya dibatasi oleh tugas, CWE, model, alat, dan bentuk serangan yang diuji. Pemeriksaan aplikasi utuh, penerapan produksi, serta serangan melalui dependensi berada di luar ruang lingkup.

I.5 Metodologi

Pengerjaan Tugas Akhir dilakukan melalui pengembangan sistem dan eksperimen terkontrol. Tahap perancangan serta implementasi dapat diulang berdasarkan hasil pengujian pada data pengembangan. Sebelum evaluasi akhir, konfigurasi dan kebijakan dibekukan agar hasil pembandingan menggunakan kondisi yang jelas. Tahap pengerjaan meliputi hal berikut.

1. **Studi literatur dan analisis kebutuhan.** Tahap ini menelaah keamanan kode hasil LLM, analisis statis, kerja sama agen, perbaikan kode, dan manipulasi reviewer. Hasil telaah digunakan untuk menentukan kebutuhan gate, bukti yang harus tersedia, kondisi kegagalan, serta pembanding yang sesuai.
2. **Penyiapan data dan skenario evaluasi.** Tahap ini membekukan versi dataset, memeriksa identitas dan duplikasi sampel, serta memisahkan data pengembangan dari data uji. Label acuan diperiksa terpisah dari keputusan alat di dalam gate. Sampel rentan dan tidak rentan disiapkan, bersama pasangan kode dengan dan tanpa komentar manipulatif. Skenario juga mencakup bukti yang bertentangan, respons agen tidak valid, dan pengujian yang tidak berjalan.
3. **Perancangan dan implementasi sistem.** Tahap ini menyusun aliran pemeriksaan, format bukti, kebijakan arbiter, dan hubungan antaragen. Implementasi mencakup sanitizer, pemanggilan SAST, penyiapan serta eksekusi pengujian, penyimpanan hasil, dan alur perbaikan. Setiap patch diperiksa ulang dengan bukti yang terkait pada versi kode terbaru.
4. **Pengujian dan pembandingan.** Tahap ini membandingkan generator tanpa gate, SAST saja, SAST dengan satu reviewer, dan gate lengkap pada tugas yang sama. Pengaruh komponen diuji dengan menonaktifkan sanitizer serta mengganti keputusan arbiter dengan keputusan reviewer, sementara bukti pemeriksaan lainnya dipertahankan. Generator dipertahankan sama, sedangkan batas revisi disamakan pada kondisi yang mencakup perbaikan. Perbedaan anggaran dicatat.
5. **Analisis hasil dan penyusunan kesimpulan.** Tahap ini menganalisis tingkat kode rentan yang memperoleh pass, penerimaan kode tidak rentan, precision, recall, F1, cakupan pemeriksaan, keberhasilan patch, dan regresi fungsi atau keamanan. Keberhasilan serangan diukur dari perubahan menjadi pass pada kode rentan yang semula ditahan. Seluruh kegagalan, token, waktu pemrosesan, serta ketidakpastian hasil turut dilaporkan agar penurunan kode rentan yang lolos dapat dinilai bersama biaya dan jumlah kasus yang ditahan.

I.6 Sistematika Pembahasan

Laporan Tugas Akhir disusun dalam lima bab dengan pembahasan sebagai berikut.

Bab I Pendahuluan menjelaskan latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan sebagai dasar pengerjaan Tugas Akhir.

Bab II Studi Literatur membahas keamanan kode hasil LLM, pemeriksaan dengan SAST dan LLM, sistem multi-agen, perbaikan kode, serta ketahanan reviewer. Bab ini juga membandingkan mekanisme pengambilan keputusan pada penelitian terkait dengan rancangan Evidence-Based Arbiter.

Bab III Analisis dan Perancangan Solusi menguraikan kebutuhan sistem, ruang lingkup bukti, arsitektur gate, peran agen, kebijakan arbiter, dan rancangan evaluasi.

Bab IV Implementasi dan Evaluasi menjelaskan penerapan rancangan, konfigurasi eksperimen, hasil pengujian, serta analisis perbandingan sesuai rumusan masalah.

Bab V Kesimpulan dan Saran merangkum jawaban atas rumusan masalah berdasarkan hasil evaluasi, menyatakan keterbatasan penelitian, dan memberikan saran pengembangan berikutnya.

DAFTAR PUSTAKA

Agarwal, A. (2026). *Refute-or-Promote: An Adversarial Stage-Gated Multi-Agent Review Methodology for High-Precision LLM-Assisted Defect Discovery*. https://arxiv.org/abs/2604.19049

Al Mamun, S. M. A., Islam, A., & Ameen, M. R. (2026). *Beyond QASecClaw: Agent Scaling, Verifier Feedback, and Evidence Reliability in Software Assurance*. https://arxiv.org/abs/2610.36947v1

Ameen, M. R., Alam, M. T. U., & Islam, A. (2026). *QASecClaw: A Multi-Agent LLM Approach for False Positive Reduction in Static Application Security Testing*. https://arxiv.org/abs/2605.01885

Bhatt, M., Chennabasappa, S., Nikolaidis, C., Wan, S., Evtimov, I., Gabi, D., Song, D., Ahmad, F., Aschermann, C., Fontana, L., Frolov, S., Giri, R. P., Kapil, D., Kozyrakis, Y., LeBlanc, D., Milazzo, J., Straumann, A., Synnaeve, G., Vontimitta, V., … Saxe, J. (2023). *Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models*. https://arxiv.org/abs/2312.04724v1

Nunez, A., Islam, N. T., Jha, S., & Rad, P. (2024). *AutoSafeCoder: A Multi-Agent Framework for Securing LLM Code Generation through Static Analysis and Fuzz Testing*. https://arxiv.org/abs/2409.10737v1

Sabir, B., Liu, S., Jang, S. I., Abuadbba, S., Gao, Y., Moore, K., Kim, S., Kim, H., & Nepal, S. (2026). *Does Teaming-Up LLMs Improve Secure Code Generation? A Comprehensive Evaluation with Multi-LLMSecCodeEval*. https://arxiv.org/abs/2603.22717

Siddiq, M. L., & Santos, J. C. S. (2022). SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques. *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 29–33. https://doi.org/10.1145/3549035.3561184

Thornton, S. (2026). *Can Adversarial Code Comments Fool AI Security Reviewers – Large-Scale Empirical Study of Comment-Based Attacks and Defenses Against LLM Code Analysis*. https://arxiv.org/abs/2602.16741

Tony, C., Mutas, M., Ferreyra, N. E. D., & Scandariato, R. (2023). *LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations*. https://arxiv.org/abs/2303.09384v1